DS 642: Applications of Parallel Computing

Shahriar Afkhami

Week 9:
Introduction to MPI

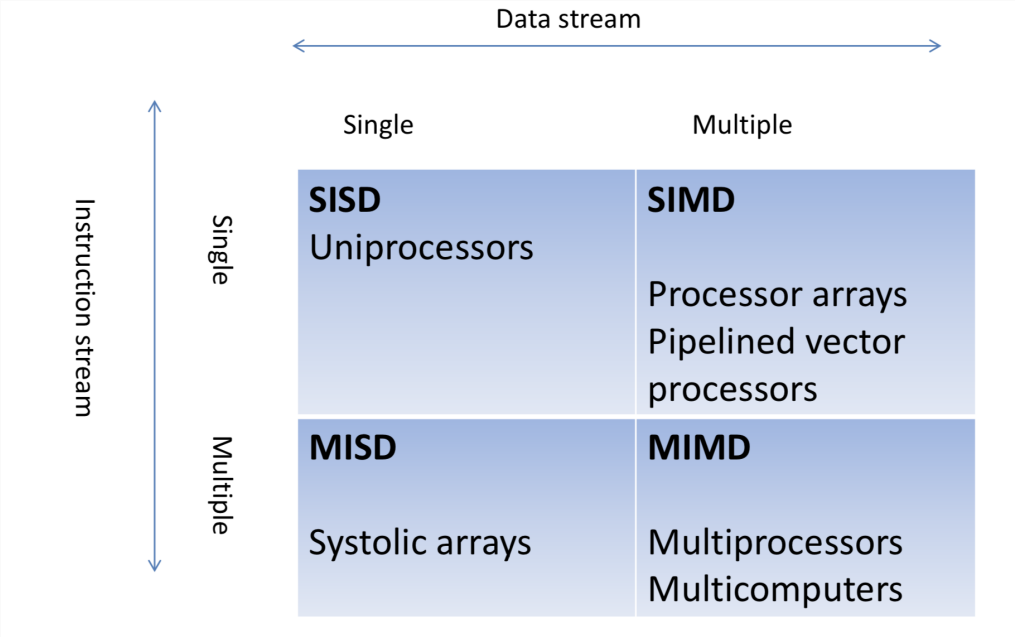
Outline

- Intro to distributed memory architectures
  - Distributed memory model
  - Message-passing programming concepts with MPI
  - Some simple examples

- Programming using MPI
  - Overview of MPI
  - Basic send and receive use
  - Blocking vs non-blocking communication
  - Collective communication

Flynn's Taxonomy

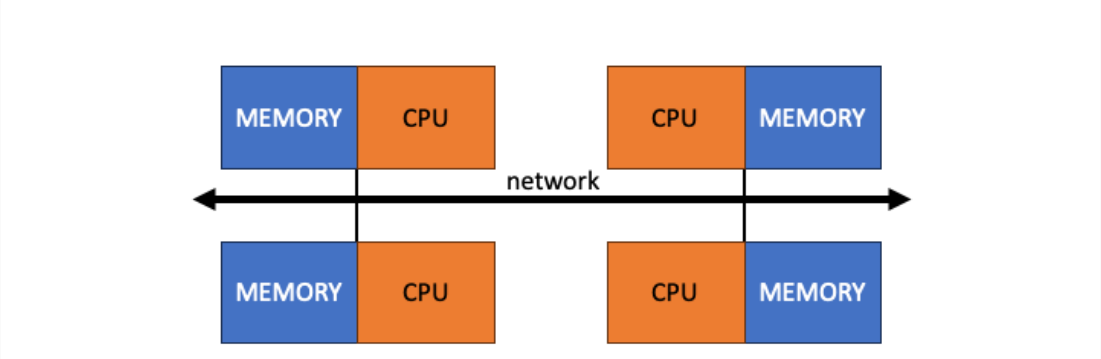
In 1966, Michael Flynn classified systems according to numbers of instruction streams and the number of data stream.



- SISD Machine example: single CPU serial computers
- MIMD: Most popular parallel computer architecture
  - Shared-memory MIMD machine
  - Distributed MIMD machine

Distributed-Memory MIMD Machine

Data exchange is through message passing:



How much does a message cost?

- *Latency*: time to get between processors (delay between send and receive times)
  - Latency is key for programs with many small messages
- *Bandwidth*: data transferred per unit time
  - Bandwidth is important for applications with mostly large messages
- How does *contention* affect communication?

Characteristics of a Network

Several features characterize an interconnect:

- *Topology*: how things are connected?
  - Crossbar; ring; 2-D, 3-D, higher-D mesh or torus; hypercube; tree; butterfly; perfect shuffle, dragon fly, ...
- *Routing*: how do we get from A to B (to reduce network congestion)?
  - Example in 2D torus: all east-west then all north-south (avoids deadlock)
- *Switching*: mapping from input to output ports
  - Circuit switching: full path reserved for entire message, like the telephone
  - Packet switching: message broken into separately-routed packets, like the post office, or internet
- *Flow control*: how do we manage limited resources?
  - Stall, store data temporarily in buffers, re-route data to other nodes, tell source node to temporarily halt, discard, etc.

Network Analogy

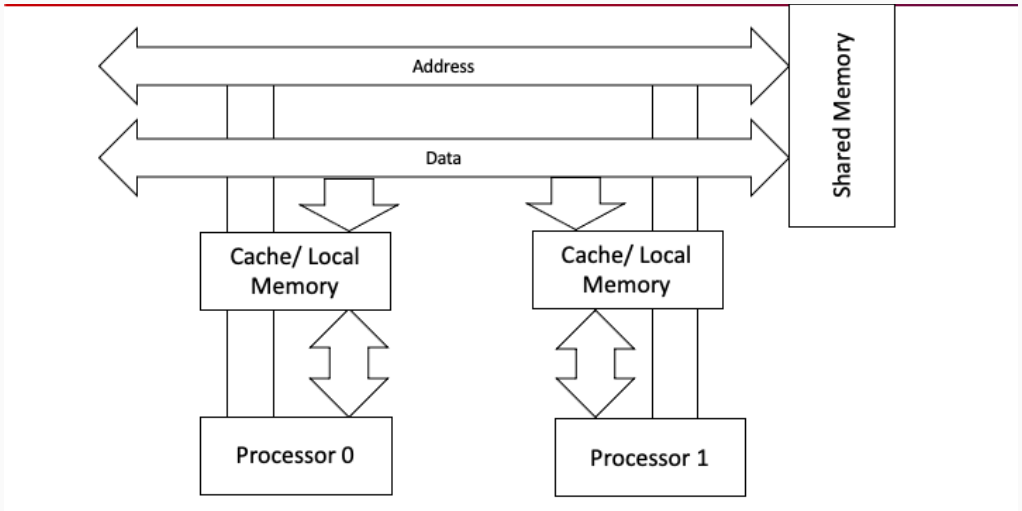
Networks are like streets

- Links = streets
- Switches = intersections
- Hops (Distances) = number of blocks traveled
- Routing algorithm = travel plan
- Stop lights are like flow control
- Short packets are like cars, long ones like buses

Properties:

- Latency: how long to get between nodes in the network.
- Bandwidth: how much data can be moved per unit time.

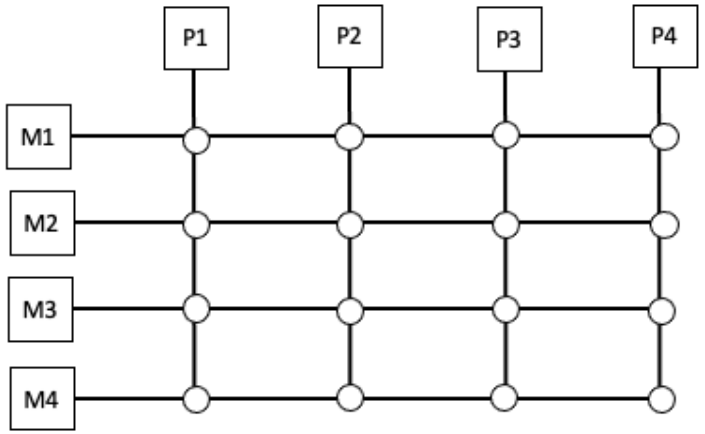
Bus topology



- One set of wires (the bus)
- Only one processor allowed at any given time
  - *Contention* for the bus is an issue
- Example: basic Ethernet, some SMPs

To have a large number of different transfers occurring at once, you need a large number of distinct wires, not just a bus, as in shared memory

Crossbar



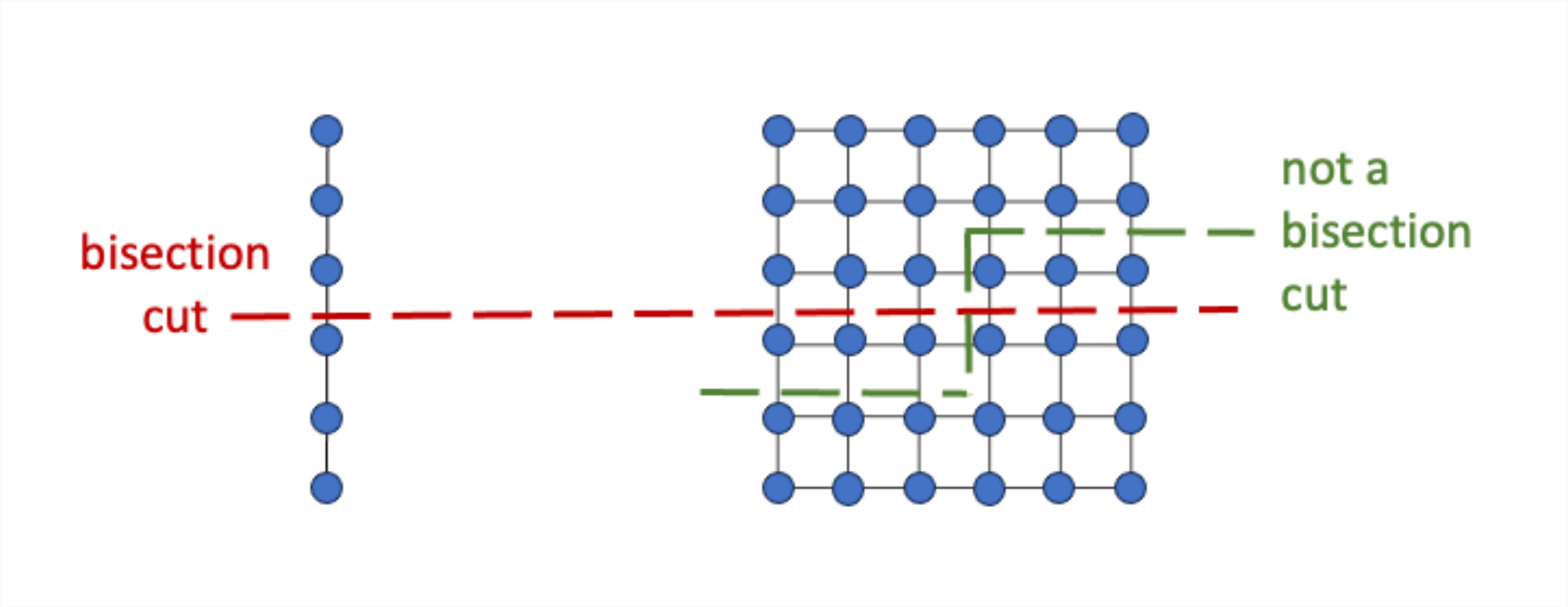
- Dedicated path from every input to every output, allowing simultaneous communication, so they are much faster than buses.
  - Takes $O(p^2)$ switches and wires!

Performance Properties of a Network

Consider latency (delay between send and receive times) and bandwidth (number of wires/time-per-bit) as metrics:

- *Diameter*: the maximum (over all pairs of nodes) of the shortest path between a given pair of nodes - indication of maximum delay that a message will encounter in being communicated between a pair of nodes.
- *Bisection bandwidth*: bandwidth across smallest cut that divides network into two equal halves - minimum volume of communication allowed between any two halves of the network; particularly important for all-to-all communication (all processors need to communicate with all others)

Performance Properties of a Network

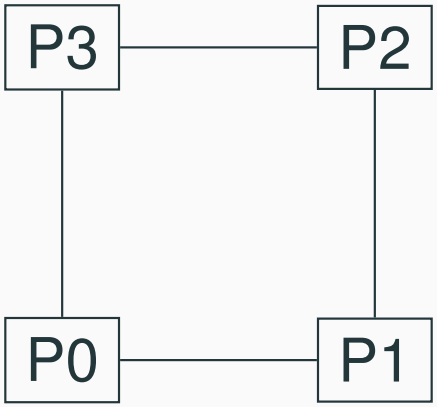


Bandwidth across “narrowest” part of the network

- Bisection bandwidth = link bandwidth
- Bisection bandwidth = link bandwidth * $\sqrt{P}$

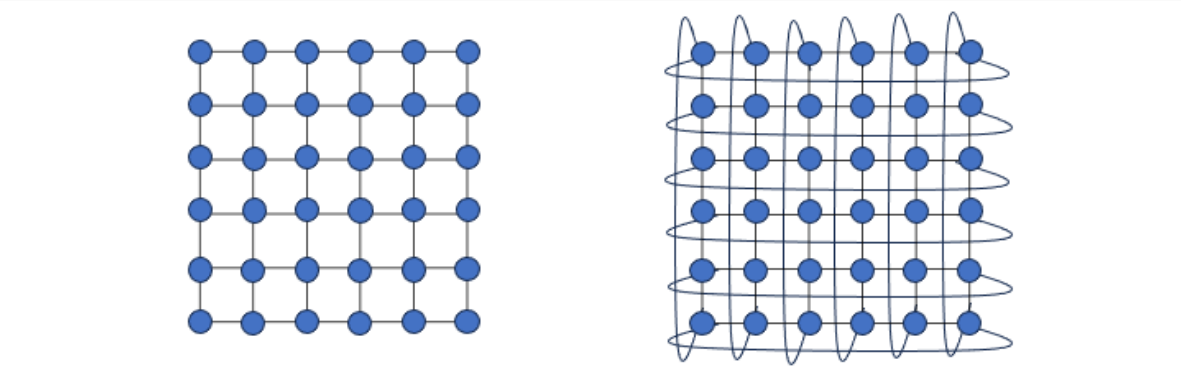
Linear and Ring Topologies

- $p - 1$ links
- Diameter $p - 1$
- Bisection bandwidth 1



- p links
- Diameter $p/2$
- Bisection bandwidth 2

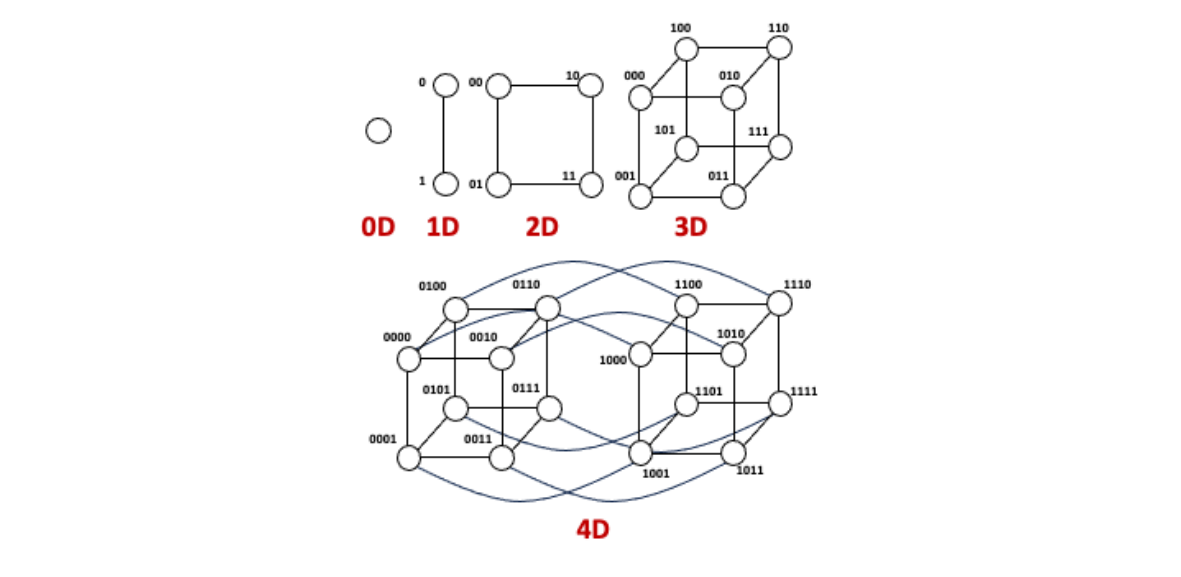
2D Mesh and torus



- 2D mesh:
  - Diameter = $2(\sqrt{P} - 1)$
  - Bisection bandwidth = $\sqrt{P}$
- 2D torus:
  - Diameter = $\sqrt{P}$
  - Bisection bandwidth = $2\sqrt{P}$

Generalizes to higher dimensions.

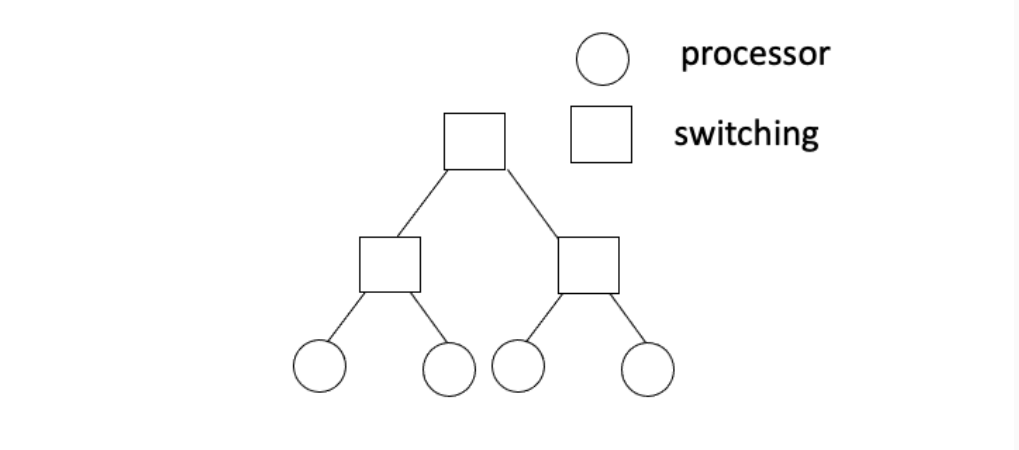
Hypercube

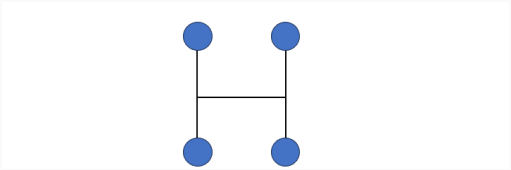


- Bisection bandwidth = $P/2$

A hypercube with P nodes is more expensive to construct than a toroidal mesh.

Binary tree

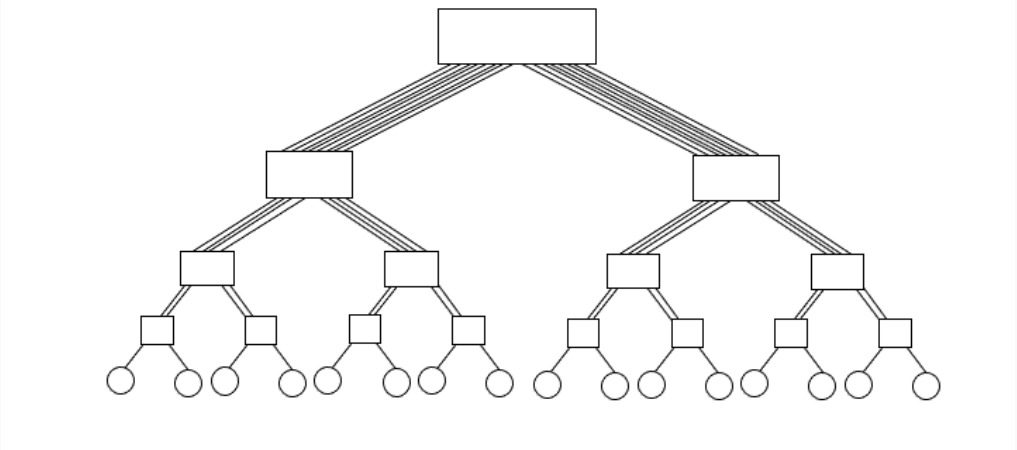




- Diameter = log P
- Bisection bandwidth = 1

Messages from one half tree to another half tree are routed through the top level nodes. Problem: links closer to root carry more traffic than those at lower levels.

Fat tree



- Processors at leaves
- Increase link bandwidth near root

Fat trees avoid bisection bandwidth problem: More (or wider) links near top.

Cost model for communicating a message, $\alpha - \beta$ model

Time to communicate a message between two nodes in a network: t_{comm}

- $t_{\text{comm}} =$ sum of the time to prepare a message for transmission and the time taken by the message to traverse the network to its destination.
- $t_{\text{s}} =$ the time required to handle a message at sending and receiving nodes.
- $\alpha =$ the time taken by the header of a message to travel between two directly connected nodes, also known as node latency. This time is directly related to the latency within the routing switch for determining which output buffer or channel the message should be forwarded to.

Cost model for communicating a message

- $\beta =$ per-word transfer time, inverse bandwidth
- $M =$ message size
- $L =$ number of communication links

Store-and-forward switching: When a message traverses a path with multiple links, each intermediate node on the path forwards the message to the next node after it has received and stored the entire message:

$$t_{\text{comm}} = t_{\text{s}} + (\alpha + \beta M)L$$

Usually $\alpha \gg \beta \gg$ time per flop. One long message is cheaper than many short ones.

Need large computation-to-communication ratio to be efficient.

Communication costs

Some basic goals:

- Prefer larger to smaller messages (avoid latency)
- Avoid communication when possible
- Overlap communication with computation

Message passing programming

Basic operations:

- Pairwise messaging: send/receive
- Collective messaging: broadcast, scatter/gather
- Collective computation: sum, max, other parallel prefix ops
- Barriers
- Environmental inquiries: How many processes are participating in this computation? Which one am I?

MPI

- Message Passing Interface: Message-Passing is a communication model used on distributed-memory architecture.
- An interface spec — many implementations, e.g. MPICH2
- MPI is not a programming language: MPI is a standard that specifies the message-passing libraries supporting parallel programming in C/C++ or Fortran.

Hello world

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    \\ reports the rank, a number between 0
    \\ and size-1, identifying the calling process
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    \\ reports the number of processes.
    printf("Hello from %d of %d\n", rank, size);
    MPI_Finalize();
    return 0;
}
```

Compilation and execution: Hello world

```
$ mpicc -g -Wall -o mpi_hello mpi_hello.c
```

Many systems also support program startup with “mpiexec”:

```
$ mpiexec -n <number of processes> ./mpi hello
```

In general, you must submit your parallel program to the compute node.

MPI Components

```
MPI_Init(&argc, &argv);
```

Its job is to create, initialize, and make available all aspects of the message passing layer. It must be called before any other MPI function.

```
MPI_Finalize();
```

Terminate MPI and clean up. It must be the last MPI function call.

Communicators

A communicator is a collection of processes that can send messages to each other. This communicator is called `MPI_COMM_WORLD`.

- Processes form *groups*
- Messages sent in *contexts*
  - Separate communication for libraries
- Group + context = communicator
- Identify process by rank in group
- Default is `MPI_COMM_WORLD`

Point-to-Point communications

Transfer message from one process to another process i.e. “send” and “receive”. Need to specify:

- What’s the data?
- What type and how much of data is sent?
- How do we identify processes?
- How does receiver identify messages?
- What does it mean to “complete” a send/recv?

MPI datatypes

Message is (address, count, datatype):

- Basic types (`MPI_INT`, `MPI_DOUBLE`)
- Contiguous arrays
- Strided arrays
- Indexed arrays
- Arbitrary structures

Users are allowed to build complex data types in MPI.

MPI datatypes

MPI datatype	C datatype
MPI_CHAR	signed **char**
MPI_SHORT	signed **short int**
MPI_INT	signed **int**
MPI_LONG	signed **long int**
MPI_LONG_LONG	signed **long long int**
MPI_UNSIGNED_CHAR	**unsigned char**
MPI_UNSIGNED_SHORT	**unsigned short int**
MPI_UNSIGNED	**unsigned int**
MPI_UNSIGNED_LONG	**unsigned long int**
MPI_FLOAT	**float**
MPI_DOUBLE	**double**
MPI_LONG_DOUBLE	**long double**
MPI_BYTE	
MPI_PACKED	

MPI tags

Messages are sent with an accompanying user-defined integer *tag*, to assist the receiving process in identifying the message

- Help distinguish different message types
- Can screen messages with wrong tag
- `MPI_ANY_TAG` is a wildcard
    - Messages can be screened at the receiving end by specifying a specific tag, or not screened by specifying `MPI_ANY_TAG` as the tag in a receive

MPI Send/Receive (Blocking)

Basic blocking point-to-point communication:

```
int
MPI_Send(void *buf, int count,
         MPI_Datatype datatype,
         int dest, int tag, MPI_Comm comm);

int
MPI_Recv(void *buf, int count,
         MPI_Datatype datatype,
         int source, int tag, MPI_Comm comm,
         MPI_Status *status);
```

The return values are error codes.

MPI send/receive semantics

- Send returns when data gets to *system*
  - ... might not yet arrive at destination!
- Recv ignores messages that don't match source and tag
  - `MPI_ANY_SOURCE` and `MPI_ANY_TAG` are wildcards
  - It can happen that one process is receiving messages from multiple processes, and the receiving process doesn't know the order in which the other processes will send the messages. Using `MPI_ANY_SOURCE` allows to receive in the order in which the processes finish.
  - There is no wildcard for specifying destination.
- Recv `status` contains more info (tag, source, size)
  - `MPI_Status` structure contains at least three members:
    - `status.MPI_SOURCE`
    - `status.MPI_TAG`
    - `status.MPI_ERROR`

Blocking send/receive restrictions

- Source, tag, and comm must match those of a pending message for the message to be received.
- Wildcards can only be used for source and tag, but not communicator.
- An error will be returned if the message buffer exceeds that allowed for by the receive.
- User must make sure that the send/receive datatypes agree. If they do not, the results are not defined.
- Receiving fewer than count occurrences of datatype is OK, but receiving more is an error
- How much data am I receiving?
  - `MPI_Get_count( &status, datatype, &recvd_count )`

Message Buffering

- Definition of “completion” for `MPI_Recv()` is trivial – the data can now be used.
- Definition of “completion” for `MPI_Send()` is trickier. Completion implies that the data has been stored away such that the program is free to overwrite the send “message” buffer.
- Non-local: the data can be sent directly to the receive buffer.
- Local (buffering): the data can be stored in a local buffer (system provided or user provided), in which case the send could return before the receive is initiated.

Safe programming practices

- A safe MPI program should not rely on system buffering for success.
- Any system will eventually run out of buffer space as message sizes are increased.
- User should design proper send/receive orders to avoid deadlock

Homework assignment: Ping-pong pseudocode

Process 0:

```
for i = 1:ntrials
  send b bytes to 1
  recv b bytes from 1
end
```

Process 1:

```
for i = 1:ntrials
  recv b bytes from 0
  send b bytes to 0
end
```

Ping-pong MPI

```
void ping(char* buf, int n, int ntrials, int p)
{
    for (int i = 0; i < ntrials; ++i) {
        MPI_Send(buf, n, MPI_CHAR, p, 0,
                 MPI_COMM_WORLD);
        MPI_Recv(buf, n, MPI_CHAR, p, 0,
                 MPI_COMM_WORLD, NULL);
    }
}
```

(Pong is similar)

Ping-pong MPI

```
for (int sz = 1; sz <= MAX_SZ; sz += 1000) {
    if (rank == 0) {
        clock_t t1, t2;
        t1 = clock();
        ping(buf, sz, NTRIALS, 1);
        t2 = clock();
        printf("%d %g\n", sz,
                (double) (t2-t1)/CLOCKS_PER_SEC);
    } else if (rank == 1) {
        pong(buf, sz, NTRIALS, 0);
    }
}
```

Approximate α-β parameters of bridges-2.

Summary

Many parallel programs can be written using just these six functions:

- `MPI_Init`
- `MPI_Finalize`
- `MPI_Comm_size`
- `MPI_Comm_rank`
- `MPI_Send`
- `MPI_Recv`

Next time: non-blocking and collective operations!